Day 8: File Handling & Multithreading (17-June-2026, Wednesday)

Session Time: 9:00 AM – 4:30 PM (Lunch Break: 12:30 PM – 1:30 PM, Tea Breaks: 10:30 AM & 3:00 PM)

Session Objectives

By the end of Day 8, learners will be able to:

- Read and write text files using `FileReader`, `BufferedReader`, `FileWriter`, and `PrintWriter`.
- Understand `Serialization` and `Deserialization` to persist object state.
- Create and manage threads using `Thread` class and `Runnable` interface.
- Implement thread synchronization using `synchronized` keyword to avoid race conditions.
- Apply `wait()`, `notify()`, and `notifyAll()` for inter-thread communication.
- Solve LeetCode: *Print in Order* and *FizzBuzz Multithreaded*.
- Solve HackerRank: *Java Threads*.
- Design a **Multi-threaded Server Log Processing System** with concurrent log ingestion and file-based persistence.

Detailed Agenda (Time Slots & Activities)

Time Slot	Activity	Duration
9:00 – 9:15	Recap of Day 7 & Day 8 Overview – File I/O and Concurrency	15 min
9:15 – 9:45	FileReader & BufferedReader – Reading Text Files	30 min
9:45 – 10:15	FileWriter & PrintWriter – Writing Text Files	30 min
10:15 – 10:30	Serialization – Serializable Interface, ObjectOutputStream, ObjectInputStream	15 min
10:30 – 10:45	Tea Break	15 min
10:45 – 11:15	Thread Creation – Extending Thread Class vs Implementing Runnable	30 min

11:15 – 11:45	Thread Lifecycle – States, Sleep, Join, Daemon Threads	30 min
11:45 – 12:15	Synchronization – Race Condition, synchronized Methods & Blocks	30 min
12:15 – 12:30	Hands-on: Race Condition Demo	15 min
12:30 – 1:30	Lunch Break	60 min
1:30 – 2:00	Inter-thread Communication – wait(), notify(), notifyAll()	30 min
2:00 – 2:30	LeetCode: Print in Order & FizzBuzz Multithreaded (Approaches)	30 min
2:30 – 2:45	HackerRank: Java Threads Walkthrough	15 min
2:45 – 3:00	Tea Break	15 min
3:00 – 3:45	Corporate Use Case: Multi-threaded Server Log Processing System	45 min
3:45 – 4:15	Hands-on: File Operations + Multithreading Exercise	30 min
4:15 – 4:30	Homework & Closing	15 min

---

1. File Handling – Reading Text Files

1.1 Why File Handling?

- Persist data beyond program execution (configuration, logs, user data).
- Process batch data (CSV, JSON, XML).
- Enable inter-program communication via files.

1.2 FileReader

- Character-based stream for reading text files.
- Reads one character at a time – inefficient for large files.
- Throws `FileNotFoundException` if file doesn't exist.

1.3 BufferedReader (Recommended)

- Wraps `FileReader` for **buffered** reading (efficient).
- Reads entire lines: `readLine()` returns null at EOF.
- Supports reading arrays of characters.

1.4 Example Pattern (Try-with-Resources)

```
try (BufferedReader br = new BufferedReader(new FileReader("input.txt"))) {
    String line;
    while ((line = br.readLine()) != null) {
        System.out.println(line);
    }
} catch (IOException e) {
    System.err.println("Error reading file: " + e.getMessage());
}
```

1.5 Common Methods

Method	Description
`read()`	Reads single character (int).
`read(char[] cbuf)`	Reads into array.
`readLine()`	Reads a line of text (null at EOF).
`skip(long n)`	Skips n characters.
`ready()`	Checks if stream is ready to be read.

1.6 File Paths

- Relative path: `"data/input.txt"` (relative to working directory).
- Absolute path: `"C:/project/data/input.txt"` (Windows) or `"/home/user/data/input.txt"` (Linux).
- Use `File.separator` for platform independence.

---

2. File Handling – Writing Text Files

2.1 FileWriter

- Character-based stream for writing text files.
- Creates file if not exists; overwrites by default.
- Use `FileWriter(file, true)` for **append** mode.

2.2 PrintWriter (Recommended)

- Wraps `FileWriter` for formatted output.
- Methods: `print()`, `println()`, `printf()` (like `System.out`)

- Methods: `print()`, `println()`, `printf()` (like `System.out`).
- Auto-flush option available.

2.3 Example Pattern

```
try (PrintWriter pw = new PrintWriter(new FileWriter("output.txt"))) {
    pw.println("Hello, World!");
    pw.printf("Value: %.2f%n", 3.14159);
} catch (IOException e) {
    System.err.println("Error writing file: " + e.getMessage());
}
```

2.4 Important Methods

Method	Description
`write(String s)`	Writes string.
`append(CharSequence cs)`	Appends to file.
`flush()`	Forces any buffered output to be written.
`close()`	Closes stream (auto-closed with try-with-resources).

---

3. Serialization

3.1 What is Serialization?

- Process of converting an object into a **byte stream** for persistence or network transfer.
- **Deserialization** is the reverse (byte stream → object).
- Allows saving entire object state (all fields).

3.2 Serializable Interface

- Marker interface (no methods) – tells JVM the class can be serialized.
- Must be implemented by the class and all its non-transient fields.
- **transient** keyword – marks fields **not** to be serialized (e.g., sensitive data, derived fields).

3.3 ObjectOutputStream & ObjectInputStream

- `ObjectOutputStream` – writes objects to a stream (file, network).
- `ObjectInputStream` – reads objects from a stream.

3.4 Example

'

```
class Employee implements Serializable {
    private static final long serialVersionUID = 1L;
    private String name;
    private transient double salary; // not serialized
}

// Serialize
try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("emp.ser"))) {
    oos.writeObject(emp);
}

// Deserialize
try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream("emp.ser"))) {
    Employee emp = (Employee) ois.readObject();
}
```

3.5 serialVersionUID

- Unique version identifier for serialized classes.
- If changed, deserialization fails with `InvalidClassException`.
- **Always declare** explicitly to avoid compatibility issues.

3.6 When to Use Serialization?

- Caching objects to disk.
- Sending objects over RMI, sockets, or REST (though JSON is more common now).
- Session management in web applications (distributed).

---

4. Thread Creation

4.1 What is a Thread?

- A thread is the smallest unit of execution within a process.
- Java supports **multithreading** – concurrent execution of multiple threads.
- Allows CPU utilization, responsiveness (UI), and parallel processing.

4.2 Two Ways to Create a Thread

4.2.1 Extending `Thread` Class

```
class MyThread extends Thread {
    public void run() {
```

```
        // Code to execute in separate thread
    }
}
MyThread t = new MyThread();
t.start(); // NOT run()
```

4.2.2 Implementing `Runnable` Interface (Preferred)

```
class MyRunnable implements Runnable {
    public void run() {
        // Code to execute
    }
}
Thread t = new Thread(new MyRunnable());
t.start();
```

4.3 Why Prefer Runnable?

- Java supports **single inheritance** – implementing Runnable allows extending another class.
- Better separation of task and execution.
- Enables use of lambda expressions (`new Thread(() -> { ... }).start()`).

4.4 Starting a Thread

- `start()` – creates a new thread and calls `run()` on it.
- **Never call `run()` directly** – that would just be a normal method call, no new thread.

---

5. Thread Lifecycle & Methods

5.1 Thread States

State	Description
NEW	Thread created but not started.
RUNNABLE	Thread is executing or ready to execute.
BLOCKED	Waiting to acquire a lock (synchronized block).
WAITING	Waiting indefinitely (wait(), join() without timeout).
TIMED_WAITING	Waiting for a specified time (sleep(), join(timeout)).
TERMINATED	Thread has completed execution.

5.2 Important Thread Methods

Method	Description
`start()`	Starts thread (transitions from NEW → RUNNABLE).
`run()`	Contains code to execute.
`sleep(long ms)`	Pauses current thread (TIMED_WAITING).
`join()`	Waits for thread to die (called from another thread).
`yield()`	Hints scheduler to let other threads run.
`interrupt()`	Interrupts a blocked/waiting thread.
`setDaemon(boolean)`	Daemon thread runs in background (e.g., garbage collector).

5.3 Daemon Threads

- Background threads that do not prevent JVM from exiting.
- Set before starting: `thread.setDaemon(true);`.
- Used for monitoring, logging, housekeeping.

---

6. Synchronization

6.1 Race Condition

- Occurs when multiple threads access shared data concurrently, and the final outcome depends on timing/interleaving.
- Example: Incrementing a counter from two threads – may miss increments due to read-modify-write interleaving.

6.2 synchronized Keyword

- Ensures **mutual exclusion** – only one thread can execute the synchronized block/method on a given object at a time.

6.2.1 Synchronized Method

```
public synchronized void increment() {
    counter++;
}
```

- Lock acquired on `this` object (or class object for static methods).

6.2.2 Synchronized Block (More Fine-Grained)

```
public void increment() {
    synchronized(this) {
        counter++;
    }
}
```

- Lock on a specific object: `synchronized(lockObj) { ... }`.

6.3 Intrinsic Lock (Monitor)

- Every object in Java has an intrinsic lock.
- When a thread enters a synchronized method/block, it acquires the lock; other threads block until it releases.

6.4 Atomic Variables (java.util.concurrent.atomic)

- `AtomicInteger`, `AtomicLong`, `AtomicReference` – provide thread-safe operations without explicit synchronization.
- `incrementAndGet()` is atomic.

6.5 Static Synchronization

- For static methods, lock is on the **Class object** (`MyClass.class`).
- Ensures only one thread executes static synchronized method across all instances.

---

7. Inter-thread Communication

7.1 Producer-Consumer Problem

- One thread produces data, another consumes it.
- Need coordination to avoid:
  - Consumer tries to consume empty buffer.
  - Producer tries to produce into full buffer.

7.2 wait(), notify(), notifyAll()

- Must be called **inside synchronized context** (on the same lock object).
- `wait()` – releases lock and puts thread in WAITING state until notified.
- `notify()` – wakes up one waiting thread (arbitrary).
- `notifyAll()` – wakes up all waiting threads.

- `notifyAll()` – wakes up all waiting threads.

7.3 Example Pattern

```
synchronized(lock) {
    while (condition) {   // Use while, not if – spurious wakeups
        lock.wait();
    }
    // Perform action
    lock.notifyAll();
}
```

7.4 Common Pitfalls

- Not checking condition in a loop (spurious wakeups).
- Forgetting to call `wait()` / `notify()` inside synchronized block.
- Using `Thread.sleep()` instead of `wait()` for coordination (does not release lock).

---

8. LeetCode Problems – Approach Discussion

8.1 Print in Order

- **Problem:** Three threads (first, second, third) should print numbers in order: first → second → third.
- **Constraints:** Only one thread runs at a time.
- **Approach 1 (Synchronized + flags):**
  - Use `synchronized` on a shared lock; maintain `state` variable (0,1,2).
  - Each method waits until its turn, then prints and advances state, notifies all.
- **Approach 2 (Semaphore):**
  - Use two `Semaphore` objects: `s1` with 1 permit (first runs first), `s2` with 0 permits.
  - First releases s1 after print; second waits on s1, releases s2; third waits on s2.
- **Approach 3 (CountDownLatch):**
  - Two latches: latch1 (for second), latch2 (for third).
- **Key concept:** Inter-thread coordination.

8.2 FizzBuzz Multithreaded

- **Problem:** Four threads (Fizz, Buzz, FizzBuzz, Number) handle numbers 1 to n.
  - Number prints numbers not divisible by 3 or 5.
  - Fizz prints "fizz" for multiples of 3.
  - Buzz prints "buzz" for multiples of 5.
  - FizzBuzz prints "fizzbuzz" for multiples of 15.

- FizzBuzz prints "FizzBuzz" for multiples of 15.
- **Approach:**
  - Shared `current` counter (1..n).
  - Each thread checks if current is its turn (using modulo).
  - Use `synchronized` or `ReentrantLock` + `Condition`.
  - `wait()` / `notifyAll()` to notify next thread after printing.
- **Key concept:** Thread coordination + conditional execution.

---

9. HackerRank: Java Threads

- **Task:** Create two threads that print numbers 1..n in sequence? (Simpler variants).
- **Typical problem:** Print "Hello" from one thread and "World" from another alternately.
- **Skills practiced:** Thread creation, `start()`, `run()`.
- **Variation:** Print numbers 1..10 using two threads – odd/even.

---

10. Corporate Use Case: Multi-threaded Server Log Processing System

10.1 Problem Context

A web server generates millions of log entries per hour. Requirements:

- Multiple server instances write logs to the same log file (concurrent writes).
- A separate log processing pipeline reads logs, filters, and stores structured data.
- Need to avoid race conditions when multiple threads write logs.
- Ability to rotate log files daily (rename and compress old logs).
- Process log entries in batches and serialize structured logs for offline analysis.

10.2 Application of Day 8 Topics

Requirement	Java Feature Used
Write logs from multiple threads safely	`synchronized` method or block on log file writer
Read logs line by line	`BufferedReader.readLine()`
Write logs to file with formatting	`PrintWriter.println()` / `printf()`
Persist processed log objects (structured)	`Serialization` – store `LogEntry` objects
Compress/archive old logs (not shown but	

Compress/archive old logs (not shown but related)	File I/O + external library
Process logs in background while server writes	Separate thread (daemon) for processing
Producer-Consumer pattern	`wait()` / `notify()` or `BlockingQueue` (advanced)
Simulate multiple server threads	Multiple `Runnable` instances writing logs
Stop processing thread gracefully	`interrupt()` and `Thread.interrupted()` check
Ensure log file is flushed after write	`flush()` or auto-flush with PrintWriter

10.3 Example Architecture

```
[Server Thread 1] ─┐
[Server Thread 2] ─┼→ [Synchronized LogWriter] → log.txt
[Server Thread 3] ─┘

[Background LogProcessor Thread] → reads log.txt → parses → writes processed.ser
```

10.4 Synchronized LogWriter Implementation (Conceptual)

```
public class LogWriter {
    private PrintWriter writer;

    public synchronized void writeLog(String message) {
        writer.println(System.currentTimeMillis() + " " + message);
        writer.flush();
    }
}
```

- `synchronized` ensures only one thread writes at a time.

10.5 LogProcessor Thread (Conceptual)

- Reads log file in a loop, processes new lines.
- Could use `tail -f` like behavior: read file, check new content periodically.
- Serializes `LogEntry` objects for analytics.

10.6 Discussion Questions

- Why use `synchronized` instead of `ConcurrentHashMap` for file writing? (File is a shared resource; synchronization ensures consistent file writes.)

- How would you make the system scalable with multiple log files? (Assign each server thread its own log file, then merge later.)
- What if the log file grows too large? (Implement log rotation – rename and create new file.)
- Why use serialization? (To store structured log objects with timestamps, severity, etc., for later analysis.)
- How to handle JVM crash – are logs lost? (Flush frequently; use buffered writer with auto-flush.)

---

11. Hands-on Exercises (Without Code Lines)

11.1 File Reading – Count Words

Describe the steps to read a text file and count the number of words (split by spaces).

11.2 File Writing – Append Mode

Explain how to add a new line to an existing file without overwriting.

11.3 Serialization – Student Records

Design a `Student` class with `name`, `rollNo`, `transient password`. Describe how to serialize and deserialize.

11.4 Thread Creation – Two Ways

Explain the difference between extending `Thread` and implementing `Runnable`. Which is better and why?

11.5 Race Condition Simulation

Given shared `int counter` incremented by two threads, describe the interleaving that causes lost updates.

11.6 wait/notify – Producer Consumer

Describe a scenario where a producer thread adds items to a list and a consumer removes them. How do `wait()` and `notify()` coordinate them?

---

12. Quiz & Quick Check (Conceptual)

1. Which class is recommended for reading text files efficiently?

- (a) `FileReader`
- (b) `BufferedReader`
- (c) `Scanner`
- (d) `ObjectInputStream`

2. The `transient` keyword is used to:

- (a) Make a field volatile
- (b) Prevent a field from being serialized
- (c) Make a field thread-safe
- (d) Mark a field as final

3. Which method must be called to start a new thread?

- (a) `run()`
- (b) `start()`
- (c) `init()`
- (d) `execute()`

4. What does the `synchronized` keyword ensure?

- (a) Threads run in parallel
- (b) Only one thread executes the block at a time
- (c) Threads are daemon threads
- (d) Threads are interrupted

5. Which method releases the lock on an object?

- (a) `sleep()`
- (b) `yield()`
- (c) `wait()`
- (d) `join()`

Answers: 1-b, 2-b, 3-b, 4-b, 5-c

---

13. Additional Resources & Cheat Sheets

- **File I/O flowchart** – Reader/Writer vs InputStream/OutputStream.
- **Serialization checklist:** Implement Serializable, define serialVersionUID, mark transient fields.
- **Thread lifecycle diagram** – states and transitions.
- **Synchronization patterns:** Synchronized method vs block, static synchronization.

---

14. Closing Summary

Day 8 combined two critical areas: file I/O for persistence and multithreading for concurrency. Learners mastered reading/writing text files using `BufferedReader` and `PrintWriter`, serialized objects for structured persistence, and created threads using both `Thread` and `Runnable`. Synchronization with `synchronized` prevented race conditions, while `wait()` / `notify()` enabled inter-thread communication. The Multi-threaded Server Log Processing System case study demonstrated how these skills integrate: concurrent log writing with synchronization, background log processing using a separate thread, and object serialization for offline analysis. LeetCode problems reinforced thread coordination techniques.

Key Takeaways:

- Use `BufferedReader` and `PrintWriter` for efficient text I/O.
- `Serializable` makes object persistence possible; `transient` excludes sensitive fields.
- Two ways to create threads: extend `Thread` or implement `Runnable`.
- `synchronized` prevents race conditions on shared resources.
- `wait()` / `notify()` enable threads to communicate and coordinate.

End of Day 8